



南開大學
Nankai University

南 开 大 学

计 算 机 学 院

大数据计算及应用实验报告

推荐系统-评分预测

2212574 文雅竹

2312966 林晖鹏

2313725 张耕嘉

指导教师：杨征路

2025 年 6 月 5 日

摘要

本实验旨在通过构建推荐系统模型，预测用户对物品的评分。实验中使用了多种推荐算法，包括基线预测、协同过滤、矩阵分解和深度学习模型，并对其性能进行了全面对比分析。实验结果表明，基线预测与协同过滤相结合的混合推荐算法（Baseline-CF）在所有指标上表现最佳，尤其是在数据稀疏性较高的情况下，能够有效提升预测精度。Baseline-CF 模型在均方误差（MSE）、均方根误差（RMSE）和平均绝对误差（MAE）等指标上均优于其他模型，表现出较强的鲁棒性。此外，实验还探讨了不同相似度计算方法对模型性能的影响，其中 Jaccard 相似度表现最佳。实验结果为推荐系统模型的选择和优化提供了重要参考。

关键字：推荐系统，协同过滤，矩阵分解，深度学习，Baseline-CF，Jaccard 相似度

目录

一、 实验内容	1
二、 问题描述	1
三、 方法原理	1
(一) 基线预测 + 协同过滤的混合推荐算法 (Baseline-CF)	1
1. 原理解析	2
2. 算法流程及伪代码	2
3. 一些实现细节	4
(二) 基线预测 (Baseline)	4
(三) 协同过滤 (CF)	5
1. 基于用户的协同过滤 (User-Based CF)	5
2. 基于物品的协同过滤 (Item-Based CF)	7
3. 相似度计算	8
4. Slope One 协同过滤	12
(四) 矩阵分解 (MF)	13
1. 矩阵奇异值分解 (SVD)	13
2. 增强奇异值分解 (SVD++)	14
3. Funk 奇异值分解 (FunkSVD)	15
(五) 深度学习	15
1. NFC	15
2. DeepFM	18
四、 实验结果及分析	20
(一) 模型性能对比	20
(二) 在 Baseline-CF 方法中不同相似度对比	20
(三) Baseline-CF 方法时间分析	22
(四) Baseline-CF 方法内存分析	23
(五) 内存占用分布	23
五、 总结	23

一、 实验内容

本实验旨在通过构建推荐系统模型，预测用户对物品的评分。实验使用的数据集包括：

- **Train.txt**：训练数据集，记录了若干用户对物品的已知评分信息；
- **Test.txt**：测试数据集，包含若干用户-物品对，需要对其评分进行预测；
- **ResultForm.txt**：结果格式模板，规定了最终预测结果的提交格式；
- **DataFormatExplanation.txt**：数据格式说明文档，详细描述了各文件的字段定义和组织结构。

实验过程中，学生可自由选择任意推荐算法实现评分预测，如基于邻域的方法（协同过滤）、基于模型的方法（矩阵分解、深度学习等），或混合推荐方法等。

二、 问题描述

在本实验中，我们面临的核心任务是：依据训练集中已有的用户-物品评分记录，构建一个推荐模型，对测试集中出现的用户-物品对进行评分预测。

具体而言，设训练集中包含若干三元组 (u, i, r_{ui}) ，表示用户 u 对物品 i 的评分为 r_{ui} 。测试集中仅给出 (u, i) 对，需要预测其对应的评分 $\hat{r}_{ui}$ ，并输出至结果文件中。

本实验的主要挑战包括但不限于：

- 数据稀疏性问题：大多数用户仅对少量物品评分，导致评分矩阵较为稀疏；
- 模型选择与调优问题：需要合理选择适合的数据建模方式，并对模型参数进行调整以获得较优预测精度；
- 性能评估问题：需使用 RMSE（均方根误差）、MAE（平均绝对值误差）等指标对模型的预测性能进行量化；
- 实验效率问题：需要在可接受的训练时间与资源消耗下完成大规模数据的建模与预测任务。

三、 方法原理

本节介绍很多方法的原理以及代码实现的思路，在本节的开始，我们先介绍我们的最优模型 **Baseline-CF**，然后在从简单到复杂介绍一下我们实现的诸多方法，对本题数据集的尝试。在第四节中有这些所有方法的测试结果。详情代码见文件夹源码中。

（一） 基线预测 + 协同过滤的混合推荐算法（Baseline-CF）

Baseline-CF 是一种将基线预测（Baseline Prediction）与协同过滤（Collaborative Filtering）相结合的混合推荐算法，融合了两种方法的优点，能有效提高推荐系统的准确性。以上两种算法详情可以先看原理后面部分。

1. 原理解析

基线预测主要捕获评分数据中的全局趋势和一阶局部性，能够有效处理评分数据中的宏观模式，但无法捕获用户-物品之间的相互作用和细粒度偏好。而相比之下，协同过滤专注于捕获更精细的局部性模式，但协同过滤在处理稀疏数据时可能缺乏足够信息，尤其是对于冷启动用户或物品，其预测可能不够稳定。

而本方法的核心思想是先使用基线预测捕获评分数据中的全局效应和用户/物品偏置，然后在此基础上应用协同过滤技术来捕获更精细的用户-物品交互模式。预测评分的公式如下：

$$\hat{r}_{ui} = \underbrace{\mu + b_u + b_i}_{\text{基线预测部分}} + \underbrace{\text{CF}(u, i, R^*)}_{\text{协同过滤部分}}$$

其中：

- $\hat{r}_{ui}$ 是用户 u 对物品 i 的预测评分
- μ 是全局平均评分
- b_u 是用户 u 的用户偏好
- b_i 是物品 i 的物品偏好
- R^* 是去除基线效应后的残差评分矩阵
- $\text{CF}(u, i, R^*)$ 是基于残差评分的协同过滤预测

2. 算法流程及伪代码

具体算法流程如下：

1. 基线模型训练：首先使用梯度下降法，基于训练数据学习基线预测模型的参数 (μ, b_u, b_i) 。
2. 残差矩阵计算：对于每个已知的评分 r_{ui} ，计算去除基线效应后的残差：

$$r_{ui}^* = r_{ui} - (\mu + b_u + b_i)$$

3. 协同过滤应用：基于残差矩阵 R^* 训练协同过滤模型，可以是基于用户或基于物品的 CF 方法，也可以是矩阵分解类方法；这里我们采用了效果比较好的基于物品的协同过滤。
4. 预测融合：对于需要预测的评分，将基线预测和协同过滤预测相加得到最终预测：

$$\hat{r}_{ui} = (\mu + b_u + b_i) + \hat{r}_{ui}^*$$

其中 $\hat{r}_{ui}^*$ 是协同过滤基于残差矩阵做出的预测。

Algorithm 1 Baseline-CF: 基于基线预测的协同过滤混合推荐算法

```

1: procedure 训练 ( $\mathcal{D}_{train}$ ,  $k$ ,  $sim\_method$ ,  $shrinkage$ ,  $n\_epochs$ ,  $\lambda$ ,  $\eta$ )
2:   // 计算用户和物品的平均评分
3:   for 每个用户  $u$  do
4:      $\bar{r}_u \leftarrow$  用户  $u$  的平均评分
5:   end for
6:   for 每个物品  $i$  do
7:      $\bar{r}_i \leftarrow$  物品  $i$  的平均评分
8:   end for
9:    $\mu \leftarrow$  全局平均评分
10:  // 训练基线预测模型
11:  初始化用户偏置  $b_u \leftarrow 0$  和物品偏置  $b_i \leftarrow 0$ 
12:  for  $epoch = 1$  to  $n\_epochs$  do
13:    for 每条评分记录  $(u, i, r_{ui}) \in \mathcal{D}_{train}$  do
14:       $\hat{r}_{ui} \leftarrow \mu + b_u + b_i$ 
15:       $e_{ui} \leftarrow r_{ui} - \hat{r}_{ui}$ 
16:       $b_u \leftarrow b_u + \eta \cdot (e_{ui} - \lambda \cdot b_u)$ 
17:       $b_i \leftarrow b_i + \eta \cdot (e_{ui} - \lambda \cdot b_i)$ 
18:    end for
19:  end for
20:  // 计算物品相似度矩阵
21:   $S \leftarrow$  指定对应相似度计算
22:  return 训练好的模型  $(S, \mu, b_u, b_i, \bar{r}_u, \bar{r}_i)$ 
23: end procedure

24: procedure 预测 ( $u, i$ , 模型,  $k$ ,  $normalize$ )
25:  获取用户  $u$  评过分的物品集合  $I_u$ 
26:  if 用户  $u$  或物品  $i$  不在训练集中 or  $I_u$  为空 then
27:    return  $\mu + b_u + b_i$  ▷ 直接用基线预测
28:  end if
29:  获取物品  $i$  与用户评分物品的相似度:  $s_{ij}, j \in I_u$ 
30:  选择 Top- $k$  个最相似的物品集合  $N^k(i; u)$ 
31:  if  $normalize = True$  then
32:    // 标准化评分偏差加权
33:    for 每个物品  $j \in N^k(i; u)$  do
34:       $baseline_j \leftarrow \mu + b_u + b_j$  ▷ 基线预测
35:       $dev_j \leftarrow r_{uj} - baseline_j$  ▷ 评分偏差
36:    end for
37:     $weighted\_dev \leftarrow \frac{\sum_{j \in N^k(i; u)} s_{ij} \cdot dev_j}{\sum_{j \in N^k(i; u)} |s_{ij}|}$ 
38:     $\hat{r}_{ui} \leftarrow \mu + b_u + b_i + weighted\_dev$ 
39:  else
40:    // 直接加权平均
41:     $\hat{r}_{ui} \leftarrow \frac{\sum_{j \in N^k(i; u)} s_{ij} \cdot r_{uj}}{\sum_{j \in N^k(i; u)} |s_{ij}|}$ 
42:  end if
43:   $\hat{r}_{ui} \leftarrow \max(0, \min(100, \hat{r}_{ui}))$  ▷ 限制在有效范围
44:  return  $\hat{r}_{ui}$ 
45: end procedure

```

3. 一些实现细节

在实际实现中，可以根据数据特点选择不同的协同过滤算法与基线预测结合：

- 对于基线部分，我们使用梯度下降优化用户和物品偏置项。
- 对于协同过滤部分，可以根据数据规模和稀疏度选择不同方法：
 - 数据较少时可使用 KNN 系列算法
 - 数据规模较大时可使用矩阵分解方法
 - 对于有特征信息的场景可使用基于内容的方法
- 两部分预测可以使用简单加权平均进行融合，权重可通过交叉验证确定。

Baseline-CF 作为混合推荐模型，结合了多种推荐策略的优点，在保持模型简洁性的同时有效提升了推荐性能。

(二) 基线预测 (Baseline)

Baseline 模型是一种简单而有效的推荐算法，用于预测用户对物品的评分。其核心思想是假设每个评分可以由以下三部分组成：

- 全局平均评分 μ ：所有训练数据中评分的平均值；
- 用户偏差 b_u ：表示用户 u 对评分的系统性偏高或偏低；
- 物品偏差 b_i ：表示物品 i 被整体打分偏高或偏低。

因此，Baseline 模型的预测评分公式如下：

$$\hat{r}_{ui} = \mu + b_u + b_i$$

其中， $\hat{r}_{ui}$ 表示用户 u 对物品 i 的预测评分。为了确定 b_u 和 b_i 的值，Baseline 模型通常采用最小化平方误差的目标函数，并引入正则项以防止过拟合：

$$\min_{b_u, b_i} \sum_{(u,i) \in \mathcal{K}} (r_{ui} - \mu - b_u - b_i)^2 + \lambda(b_u^2 + b_i^2)$$

其中：

- $\mathcal{K}$ 是训练数据中已知评分的用户-物品对集合；
- r_{ui} 是实际评分；
- λ 是正则化系数。

Algorithm 2 Baseline 预测模型的训练过程

Input: 训练集评分 (u, i, r_{ui}) ，学习率 η ，正则化参数 λ ，迭代轮数 n_epochs

Output: 用户偏置 b_u ，物品偏置 b_i ，全局均值 μ

- 1: 初始化：对每个用户 u 和物品 i ，设 $b_u[u] \leftarrow 0$ ， $b_i[i] \leftarrow 0$
- 2: 计算全局均值 $\mu \leftarrow$ 所有训练评分的平均值
- 3: **for** epoch $\leftarrow 1$ to n_epochs **do**
- 4: **for** 每个评分 (u, i, r_{ui}) **do**

```

5:      $\hat{r}_{ui} \leftarrow \mu + b_u[u] + b_i[i]$ 
6:      $e_{ui} \leftarrow r_{ui} - \hat{r}_{ui}$ 
7:      $b_u[u] \leftarrow b_u[u] + \eta \cdot (e_{ui} - \lambda \cdot b_u[u])$ 
8:      $b_i[i] \leftarrow b_i[i] + \eta \cdot (e_{ui} - \lambda \cdot b_i[i])$ 
9:   end for
10: end for

```

(三) 协同过滤 (CF)

协同过滤是推荐系统的核心技术之一，其核心思想是通过用户与物品的历史交互行为（如评分、点击、购买等），发现用户或物品之间的相似性，并基于此预测未知的偏好。

协同过滤基于一个假设：相似的用户对物品的偏好相似，相似的物品会被相似用户偏好。基于此，协同过滤可以分为基于用户的协同过滤 (User-Based CF) 和基于物品的协同过滤 (Item-Based CF)。

KNN 算法是一种基于实例的机器学习算法，核心思想是通过计算样本间的相似性（如余弦相似度、皮尔逊相关系数等），找到当前样本的 K 个最近邻，利用近邻的特征或标签进行预测或分类。

KNN with Means 是 KNN 的改进版本，核心是在计算相似性或预测评分时引入均值中心化 (Mean Centering)，以消除用户或物品的评分偏差。常见于用户协同过滤场景。

在此次推荐系统实验的协同过滤方法中，使用了 KNN 与 KNN with means 两种方法进行基于用户或物品的相似性进行推荐。

1. 基于用户的协同过滤 (User-Based CF)

基于“人以类聚”的理念，当为用户 A 进行个性化推荐时，先找出与用户 A 兴趣相似的其他用户集合，然后把这些相似用户喜欢但用户 A 尚未接触过的物品推荐给 A。

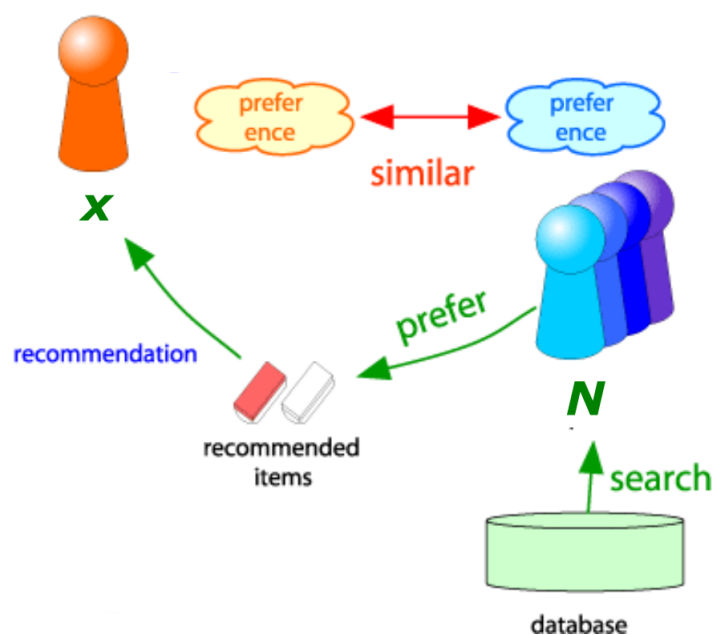


图 1: user-user

- 算法步骤

1. 计算用户之间的相似度（如余弦相似度、皮尔逊相关系数）。
2. 找出目标用户的“最近邻”（最相似的 Top-K 用户）。
3. 根据邻居用户对目标物品的评分加权平均，预测目标用户的评分。

- 计算公式 (KNN)

$$\hat{r}_{ui} = \frac{\sum_{v \in N_u^k(i)} \text{sim}(u, v) \cdot r_{vi}}{\sum_{v \in N_u^k(i)} |\text{sim}(u, v)|}$$

- $\hat{r}_{ui}$: 用户 u 对物品 i 的预测评分
- $N_u^k(i)$: 与用户 u 最相似的 k 个用户集合（这些用户需对物品 i 有评分）
- $\text{sim}(u, v)$: 用户 u 与用户 v 之间的相似度
- r_{vi} : 用户 v 对物品 i 的实际评分

KNN

```

1  函数 fit(评分数据框 ratings_df):
2      // 数据准备
3      用户列表 U ← 提取ratings_df中所有唯一用户ID并排序
4      物品列表 I ← 提取ratings_df中所有唯一物品ID并排序
5
6      // 构建ID与索引的映射
7      用户索引映射 user_to_idx ← {用户ID: 索引}
8      物品索引映射 item_to_idx ← {物品ID: 索引}
9
10     // 初始化评分矩阵 R[用户数, 物品数]
11     评分矩阵 R ← 全0矩阵(行数=|U|, 列数=|I|)
12
13     // 填充评分矩阵
14     对ratings_df中的每行(用户u, 物品i, 评分r):
15         R[user_to_idx[u], item_to_idx[i]] ← r
16
17     // 计算平均值
18     用户平均评分向量 user_means ← 对R按行计算非零元素平均值
19     物品平均评分向量 item_means ← 对R按列计算非零元素平均值
20
21     // 计算相似度矩阵
22     如果方法为'user':
23         相似度矩阵 S ← 计算用户间相似度(R) // 基于用户-物品矩阵
24     否则: // 方法为'item'
25         相似度矩阵 S ← 计算物品间相似度(R^T) // 基于物品-用户矩阵

```

- 计算公式 (KNN with means)

$$\hat{r}_{ui} = \bar{r}_u + \frac{\sum_{v \in N(u)} \text{sim}(u, v) \cdot (r_{vi} - \bar{r}_v)}{\sum_{v \in N(u)} |\text{sim}(u, v)|}$$

- $\bar{r}_u$: 用户 u 的平均评分。

- $N(u)$: 相似用户集合。

KNN with means

```

1 函数 fit (评分数据框):
2     提取唯一用户和物品ID
3     构建用户/物品ID与索引的双向映射
4
5     初始化用户-物品评分矩阵
6     填充评分矩阵
7
8     计算用户平均评分向量
9     计算物品平均评分向量
10
11    创建中心化评分矩阵:
12        如果是用户协同过滤:
13            每个用户评分减去该用户平均分
14        否则(物品协同过滤):
15            每个物品评分减去该物品平均分
16
17    计算相似度矩阵:
18        如果是用户协同过滤:
19            基于中心化矩阵计算用户间相似度
20        否则:
21            基于中心化矩阵转置计算物品间相似度

```

2. 基于物品的协同过滤 (Item-Based CF)

基于“物以群分”，预先依据所有用户的历史偏好数据计算物品之间的相似性，然后把与用户喜欢的物品相类似的物品推荐给用户。

- 算法步骤
 1. 计算物品之间的相似度。
 2. 找出目标物品的相似物品集合。
 3. 根据用户对相似物品的历史评分加权平均，预测目标评分。

- 计算公式 (KNN)

$$\hat{r}_{ui} = \frac{\sum_{j \in N(i)} \text{sim}(i, j) \cdot r_{uj}}{\sum_{j \in N(i)} |\text{sim}(i, j)|}$$

- $N(i)$: 相似物品集合。

- 计算公式 (KNN with means)

$$\hat{r}_{ui} = \mu_i + \frac{\sum_{j \in N_i^k(u)} \text{sim}(i, j) \cdot (r_{uj} - \mu_j)}{\sum_{j \in N_i^k(u)} |\text{sim}(i, j)|}$$

- μ_i : 物品 i 的平均评分
- μ_j : 物品 j 的平均评分
- $(r_{uj} - \mu_j)$: 用户 u 对物品 j 的评分偏差

3. 相似度计算

• 余弦相似度

将用户/物品的评分向量视为空间中的向量，计算其夹角余弦值。

$$\text{sim}(u, v) = \frac{\sum_i r_{ui} \cdot r_{vi}}{\sqrt{\sum_i r_{ui}^2} \cdot \sqrt{\sum_i r_{vi}^2}}$$

- u, v : 代表不同的用户或者物品（在基于用户的协同过滤中通常为用户，在基于物品的协同过滤中通常为物品）。
- i : 表示物品（在计算用户相似度场景下）或者用户（在计算物品相似度场景下）的索引，用于遍历所有相关的评分记录。
- r_{ui} : 用户 u 对物品 i 的评分（在基于用户的协同过滤计算用户相似度时）；或者物品 i 在用户 u 处的得分情况（在基于物品的协同过滤计算物品相似度时）。
- r_{vi} : 用户 v 对物品 i 的评分（在基于用户的协同过滤计算用户相似度时）；或者物品 i 在用户 v 处的得分情况（在基于物品的协同过滤计算物品相似度时）。

计算余弦相似度的代码部分如下。代码克服了余弦相似度将缺失的评分视为“负面”的问题。

代码中计算余弦相似度时使用了 $\text{mask} = (\text{matrix}[i] > 0) \ \& \ (\text{matrix}[j] > 0)$ 来确定共同评分的物品（或用户）。这意味着它只考虑了在两个实体（用户或物品）中都存在且大于 0 的评分。

对于没有评分的物品/用户，它们对应的矩阵位置的值通常为 0（初始化时为 0），并且在 mask 计算中会被排除掉，不会参与到点积和范数的计算中。

cosine Similarity

```

1  函数 计算余弦相似度(矩阵 M):
2      n ← M的行数
3      相似度矩阵 S ← 创建全0矩阵(n×n)
4
5      对 i 从 0 到 n-1:
6          对 j 从 i 到 n-1:
7              共同评分索引集 C ← {k | M[i,k] > 0 且 M[j,k] > 0}
8
9              如果C不为空:
10                 向量a ← M[i, C]
11                 向量b ← M[j, C]
12                 S[i, j] ← a · b / (||a|| · ||b||)
13                 S[j, i] ← S[i, j]  // 对称性复制
14
15  返回 S
  
```

• 皮尔逊相关系数

衡量两个变量之间的线性相关程度，计算两个变量的协方差除以标准差的乘积。

$$\text{sim}(u, v) = \frac{\sum_i (r_{ui} - \bar{r}_u)(r_{vi} - \bar{r}_v)}{\sqrt{\sum_i (r_{ui} - \bar{r}_u)^2} \cdot \sqrt{\sum_i (r_{vi} - \bar{r}_v)^2}}$$

除之前参数外，新增参数含义如下：

- $\bar{r}_u$: 用户 u 的平均评分, 用于对用户 u 的评分进行均值中心化处理。
- $\bar{r}_v$: 用户 v 的平均评分, 用于对用户 v 的评分进行均值中心化处理。

pearson Similarity

```

1  函数 计算皮尔逊相似度(矩阵 M):
2      n ← M的行数
3      初始化相似度矩阵 S[n×n]
4
5      对 i 从 0 到 n-1:
6          对 j 从 i 到 n-1:
7              共同评分索引集 C ← {k | M[i,k] > 0 且 M[j,k] > 0}
8              如果 |C| > 1:
9                  a ← M[i,C]
10                 b ← M[j,C]
11                 如果 std(a)=0 或 std(b)=0:
12                     S[i,j] ← 0
13                 否则:
14                     S[i,j] ← corrccoef(a,b)
15                 S[j,i] ← S[i,j] // 对称性
16
17     返回 S

```

• 皮尔逊-基线相关系数

这种方法是在计算皮尔逊相关系数之前, 先从评分中减去基于基线(例如用户的平均评分或物品的平均评分)的预测值。这试图移除用户或物品的整体评分偏差, 从而可能更准确地衡量用户或物品之间真正的相似性。

$$\text{pearson_baseline_sim}(u, v) = \hat{\rho}_{uv} = \frac{\sum_{i \in I_{uv}} (r_{ui} - b_{ui}) \cdot (r_{vi} - b_{vi})}{\sqrt{\sum_{i \in I_{uv}} (r_{ui} - b_{ui})^2} \cdot \sqrt{\sum_{i \in I_{uv}} (r_{vi} - b_{vi})^2}}$$

or

$$\text{pearson_baseline_sim}(i, j) = \hat{\rho}_{ij} = \frac{\sum_{u \in U_{ij}} (r_{ui} - b_{ui}) \cdot (r_{uj} - b_{uj})}{\sqrt{\sum_{u \in U_{ij}} (r_{ui} - b_{ui})^2} \cdot \sqrt{\sum_{u \in U_{ij}} (r_{uj} - b_{uj})^2}}$$

- u, v : 代表不同的用户 (User), 用于计算用户之间的 Pearson - baseline 相似度。
- i, j : 代表不同的物品 (Item), 用于计算物品之间的 Pearson - baseline 相似度。
- r_{ui} : 用户 u 对物品 i 的原始评分 (rating)。
- b_{ui} : 用户 u 对物品 i 的基准评分 (baseline rating), 通常是通过一定的基准预测模型(比如考虑用户偏差、物品偏差等)得到的评分估计值, 用于对原始评分进行修正。
- I_{uv} : 同时被用户 u 和用户 v 评分过的物品集合 (Item set rated by both user u and user v), 是计算用户 u 和 v 之间相似度时所依据的共同评分物品范围。
- U_{ij} : 同时对物品 i 和物品 j 评过分的用户集合 (User set who rated both item i and item j), 是计算物品 i 和 j 之间相似度时所依据的共同评分用户范围。

- $\hat{\rho}_{uv}$: 用户 u 和用户 v 之间经过基准修正后的 Pearson 相关系数 (即 Pearson - baseline 相似度值), 衡量用户 u 和 v 评分行为的相似程度。
- $\hat{\rho}_{ij}$: 物品 i 和物品 j 之间经过基准修正后的 Pearson 相关系数 (即 Pearson - baseline 相似度值), 衡量物品 i 和 j 获得评分模式的相似程度。

pearson-baseline Similarity

```

1  函数 计算皮尔逊基线相似度(矩阵 M, 用户均值 UM, 物品均值 IM, 方法 type):
2      n ← M的行数
3      初始化对称矩阵 S[n×n]
4
5      对 i 从 0 到 n-1:
6          对 j 从 i 到 n-1:
7              共同评分索引集 C ← {k | M[i,k] > 0 且 M[j,k] > 0}
8              if |C| > 1:
9                  a ← M[i,C]
10                 b ← M[j,C]
11
12                 // 计算基线并中心化
13                 base_i ← UM[i] (若 type=用户) 或 IM[i] (若 type=物品)
14                 base_j ← UM[j] (若 type=用户) 或 IM[j] (若 type=物品)
15                 a_c ← a - base_i
16                 b_c ← b - base_j
17
18                 // 计算相似度
19                 if std(a_c)=0 或 std(b_c)=0:
20                     S[i,j] ← 0
21                 else:
22                     S[i,j] ← sum(a_c · b_c) / √(sum(a_c²) · sum(b_c²))
23                 S[j,i] ← S[i,j] // 对称性
24
25      返回 S

```

• 调整余弦相似度 (Adjusted Cosine Similarity)

该方法对每个用户的评分进行中心化, 即减去该用户的平均评分, 从而消除用户评分尺度的偏差:

$$\text{sim}(i, j) = \frac{\sum_{u \in U_{ij}} (r_{ui} - \bar{r}_u)(r_{uj} - \bar{r}_u)}{\sqrt{\sum_{u \in U_{ij}} (r_{ui} - \bar{r}_u)^2} \cdot \sqrt{\sum_{u \in U_{ij}} (r_{uj} - \bar{r}_u)^2}}$$

其中, U_{ij} 表示对物品 i 和 j 都有评分的用户集合, $\bar{r}_u$ 是用户 u 的平均评分。

adjusted cosine 相似度

```

1  normalized_matrix = np.zeros_like(matrix)
2  for u_idx in range(matrix.shape[0]):
3      user_ratings = matrix[u_idx, :]
4      rated_items = user_ratings > 0
5      if np.any(rated_items):
6          user_mean = np.mean(user_ratings[rated_items])

```

```

7         normalized_matrix[u_idx, rated_items] = user_ratings[rated_items]
          - user_mean
8
9     # 转置为物品-用户矩阵并计算相似度
10    item_user_matrix = normalized_matrix.T
11    item_similarity = cosine_similarity(item_user_matrix)

```

• IUF 加权相似度 (Inverse User Frequency Weighting)

IUF 思想类似 TF-IDF，稀有物品更具信息量，因此在评分中对用户活跃度高的项目降低权重。每个评分项乘以 $\log\left(\frac{n}{n_i}\right)$ ，其中 n_i 为评价该项目的用户数。

$$\text{IUF}(i) = \log\left(\frac{|U| + 1}{|\{u : r_{ui} > 0\}| + 1}\right)$$

IUF 本质上是一种启发式权重调整方法，其性能取决于数据是否符合“热门物品相似性无意义”的假设。在多数实际场景中，原始相似度（如余弦、皮尔逊）因直接建模特征相关性，反而更稳健。此处使用该相似度是为了让样本多样化。

在计算物品相似度前，先将评分矩阵按列乘以 IUF 权重：

IUF 加权

```

1 def compute_iuf_similarity(item_user_matrix):
2     n_items, n_users = item_user_matrix.shape
3     sim_matrix = np.zeros((n_items, n_items))
4     # 计算每个用户的评分数量
5     user_ratings_count = np.sum(item_user_matrix > 0, axis=0)
6     # 计算IUF权重 (log(N/n_u))
7     iuf_weights = np.log(n_users / (user_ratings_count + 1e-10))
8     # 应用IUF权重到评分矩阵
9     weighted_matrix = np.zeros_like(item_user_matrix)
10    for u in range(n_users):
11        mask = item_user_matrix[:, u] > 0
12        weighted_matrix[mask, u] = item_user_matrix[mask, u] * iuf_weights[u]
13    norms = np.sqrt(np.sum(weighted_matrix ** 2, axis=1))
14    norms[norms == 0] = 1e-10 # 避免除以零
15    # 步骤2: 归一化物品向量(除以其范数)
16    normalized_weighted_matrix = weighted_matrix / norms[:, np.newaxis]
17    # 步骤3: 通过矩阵乘法一次性计算所有余弦相似度
18    sim_matrix = np.dot(normalized_weighted_matrix,
        normalized_weighted_matrix.T)

```

• 杰卡德相似度 (Jaccard Similarity)

杰卡德相似度衡量两个集合之间的重叠程度，适用于二值型评分（如是否评分）。仅考虑用户是否对物品进行过评分： $\text{sim}(i, j) = \frac{|U_i \cap U_j|}{|U_i \cup U_j|}$ 其中 U_i 、 U_j 为对物品 i 、 j 评分过的用户集合。

Jaccard 相似度

```

1 item_user_matrix = matrix.T

```

```

2     n_items = item_user_matrix.shape[0]
3     # Jaccard相似度 - 仅考虑是否评分, 不考虑评分值
4     binary_matrix = (item_user_matrix > 0).astype(np.float32)
5     # 计算每个物品评分的用户数 (每行中的1的个数)
6     item Rated_counts = np.sum(binary_matrix, axis=1)
7     # 计算物品对之间的交集数量 (矩阵乘法A · B^T)
8     intersection_matrix = np.dot(binary_matrix, binary_matrix.T)
9     # 计算并集大小: A的评分数 + B的评分数 - 交集大小
10    # 使用广播机制计算所有对的并集
11    union_matrix = item Rated_counts[:, np.newaxis] + item Rated_counts -
        intersection_matrix
12    # 计算Jaccard相似度: 交集/并集
13    # 避免除以0
14    union_matrix[union_matrix == 0] = 1
15    item_similarity = intersection_matrix / union_matrix

```

4. Slope One 协同过滤

Slope One 是一种基于评分差值的协同过滤算法, 核心思想是利用用户之间对物品评分的差值来预测未评分物品的分值。

对于每一对物品 i 和 j , 定义评分偏差 $dev_{i,j}$ 与频次 $freq_{i,j}$ 为:

$$dev_{i,j} = \frac{\sum_{u \in U_{i,j}} (r_{u,i} - r_{u,j})}{|U_{i,j}|}, \quad freq_{i,j} = |U_{i,j}|$$

其中 $U_{i,j}$ 表示同时对物品 i 和 j 打分的用户集合, $r_{u,i}$ 表示用户 u 对物品 i 的评分。对于用户 u 和目标物品 t , 预测评分 $\hat{r}_{u,t}$ 的计算如下:

$$\hat{r}_{u,t} = \frac{\sum_{j \in R_u \cap I_t} freq_{t,j} \cdot (r_{u,j} + dev_{t,j})}{\sum_{j \in R_u \cap I_t} freq_{t,j}}$$

其中:

- R_u 表示用户 u 已评分的物品集合;
- I_t 表示与目标物品 t 存在偏差记录的物品集合;
- 若分母为 0, 即无有效偏差记录, 则预测值默认为一个设定的常数 $default_pred$ 。

首先将评分矩阵随机划分为训练集和验证集。训练阶段对训练数据构建偏差矩阵与频次矩阵; 验证阶段使用 RMSE (均方根误差) 对模型进行评估, RMSE 计算公式为:

$$RMSE = \sqrt{\frac{1}{N} \sum_{i=1}^N (r_i - \hat{r}_i)^2}$$

其中 r_i 是实际评分, $\hat{r}_i$ 是模型预测评分, N 是验证集样本总数。

通过在不同的参数组合下 (如 min_freq 和 $default_pred$) 训练模型并计算 RMSE, 选择使验证集 RMSE 最小的参数组合作为最优结果。

(四) 矩阵分解 (MF)

1. 矩阵奇异值分解 (SVD)

矩阵分解 (Matrix Factorization) 是一种经典的协同过滤推荐方法，其目标是从用户-物品评分矩阵中挖掘潜在的隐含因子 (Latent Factors)，以刻画用户与物品之间的相互关系。SVD (Singular Value Decomposition) 是其中一种典型实现形式。

SVD 模型通过下式对用户 u 对物品 i 的评分进行预测：

$$\hat{r}_{ui} = \mu + b_u + b_i + \mathbf{p}_u^\top \mathbf{q}_i$$

其中：

- μ 为所有评分的全局平均值；
- b_u 为用户 u 的偏置项，用于刻画用户的整体评分倾向；
- b_i 为物品 i 的偏置项，用于刻画物品的受欢迎程度；
- $\mathbf{p}_u \in \mathbb{R}^f$ 为用户 u 的潜在特征向量；
- $\mathbf{q}_i \in \mathbb{R}^f$ 为物品 i 的潜在特征向量；
- f 为潜在特征维度的数量；
- $\mathbf{p}_u^\top \mathbf{q}_i$ 则表示用户与物品之间的潜在因子交互值。

为了学习模型参数，采用如下的损失函数，该目标函数由预测误差平方和和正则项构成：

$$\mathcal{L} = \sum_{(u,i) \in \mathcal{K}} (r_{ui} - \hat{r}_{ui})^2 + \lambda (b_u^2 + b_i^2 + \|\mathbf{p}_u\|^2 + \|\mathbf{q}_i\|^2)$$

其中：

- $\mathcal{K}$ 表示所有已知评分对组成的训练集合；
- λ 为正则化系数，用于防止模型过拟合；
- r_{ui} 为用户 u 对物品 i 的真实评分。

使用随机梯度下降 (Stochastic Gradient Descent, SGD) 对模型参数进行优化。

对于训练集中未出现的用户或物品 (即冷启动情况)，模型采用退化方案，即：

$$\hat{r}_{ui} = \mu$$

表示无法获得用户或物品信息时，仅使用全局平均评分作为预测值。

模型性能受到多个超参数的影响，包括：

- 潜在因子维度 f ；
- 学习率 γ ；
- 正则化系数 λ ；
- 迭代轮数 (Epoch 数)。

因此, 我们通过网格搜索 (Grid Search) 在验证集上评估不同参数组合的性能, 选取在 RMSE 最小的配置作为最终模型参数。

Algorithm 3 SVD 矩阵分解模型训练过程

Input: 训练集评分 (u, i, r_{ui}) , 学习率 η , 正则化系数 λ , 迭代轮数 n_epochs , 潜在因子维度 k

Output: 用户偏置 b_u , 物品偏置 b_i , 用户因子矩阵 P_u , 物品因子矩阵 Q_i

```

1: 初始化: 对所有用户  $u$  和物品  $i$ , 设  $b_u[u] \leftarrow 0$ ,  $b_i[i] \leftarrow 0$ 
2: 随机初始化  $P_u[u], Q_i[i] \sim \mathcal{N}(0, 0.1)$ 
3: 计算全局评分均值  $\mu$ 
4: for epoch  $\leftarrow 1$  to  $n\_epochs$  do
5:   for 每条评分  $(u, i, r_{ui})$  do
6:      $\hat{r}_{ui} \leftarrow \mu + b_u[u] + b_i[i] + \langle P_u[u], Q_i[i] \rangle$ 
7:      $e_{ui} \leftarrow r_{ui} - \hat{r}_{ui}$ 
8:      $b_u[u] \leftarrow b_u[u] + \eta \cdot (e_{ui} - \lambda \cdot b_u[u])$ 
9:      $b_i[i] \leftarrow b_i[i] + \eta \cdot (e_{ui} - \lambda \cdot b_i[i])$ 
10:     $P_u[u] \leftarrow P_u[u] + \eta \cdot (e_{ui} \cdot Q_i[i] - \lambda \cdot P_u[u])$ 
11:     $Q_i[i] \leftarrow Q_i[i] + \eta \cdot (e_{ui} \cdot P_u[u] - \lambda \cdot Q_i[i])$ 
12:   end for
13: end for
  
```

2. 增强奇异值分解 (SVD++)

SVD++ (Singular Value Decomposition Plus Plus) 是推荐系统中的一种矩阵分解方法, 最初由 Yehuda Koren 提出, 显著提升了 Netflix Prize 比赛中的准确率。该方法在传统 SVD 基础上加入了用户的隐式反馈信息, 用于建模用户潜在兴趣。

SVD++ 的预测评分公式如下:

$$\hat{r}_{ui} = \mu + b_u + b_i + \left(\mathbf{p}_u + \frac{1}{\sqrt{|N(u)|}} \sum_{j \in N(u)} \mathbf{y}_j \right)^\top \mathbf{q}_i$$

其中:

- $\hat{r}_{ui}$: 用户 u 对物品 i 的预测评分;
- μ : 全局平均评分;
- b_u 和 b_i : 用户和物品的偏置项;
- $\mathbf{p}_u$ 和 $\mathbf{q}_i$: 用户和物品的 latent factor 向量 (显式反馈);
- $N(u)$: 用户 u 曾交互过的物品集合 (无论是否打分);
- $\mathbf{y}_j$: 表示物品 j 对用户偏好的“隐式反馈向量”;
- $\frac{1}{\sqrt{|N(u)|}} \sum_{j \in N(u)} \mathbf{y}_j$: 用户的隐式反馈向量加权和。

模型通过最小化如下正则化平方误差损失函数进行学习:

$$\mathcal{L} = \sum_{(u,i) \in \mathcal{R}} (r_{ui} - \hat{r}_{ui})^2 + \lambda \left(\|\mathbf{p}_u\|^2 + \|\mathbf{q}_i\|^2 + \sum_{j \in N(u)} \|\mathbf{y}_j\|^2 + b_u^2 + b_i^2 \right)$$

其中, λ 为正则化参数, 用于防止过拟合。

Algorithm 4 SVD++ 模型训练过程**Input:** 显式评分集合 (u, i, r_{ui}) , 隐式反馈集合 $N(u)$ **Output:** 用户偏置 b_u , 物品偏置 b_i , 因子 p_u, q_i, y_j

```

1: 初始化  $b_u, b_i \leftarrow 0$ ,  $p_u, q_i, y_j \sim \mathcal{N}(0, 0.1)$ 
2: 计算全局均值  $\mu$ 
3: for epoch = 1 to  $T$  do
4:   for 每个评分  $(u, i, r_{ui})$  do
5:      $N(u) \leftarrow$  用户  $u$  的隐式反馈集合
6:      $\hat{y} \leftarrow \frac{1}{\sqrt{|N(u)|}} \sum_{j \in N(u)} y_j$ 
7:      $x_u \leftarrow p_u[u] + \hat{y}$ 
8:      $\hat{r}_{ui} \leftarrow \mu + b_u[u] + b_i[i] + \langle q_i[i], x_u \rangle$ 
9:      $e_{ui} \leftarrow r_{ui} - \hat{r}_{ui}$ 
10:    更新  $b_u[u], b_i[i] \leftarrow$  SGD 规则
11:    更新  $p_u[u], q_i[i] \leftarrow$  SGD 规则
12:    for 每个  $j \in N(u)$  do
13:      更新  $y_j \leftarrow$  SGD 规则
14:    end for
15:  end for
16: end for

```

3. Funk 奇异值分解 (FunkSVD)

FunkSVD 是一种基于矩阵分解的协同过滤算法, 通过学习用户和物品在潜在因子空间中的向量表示, 进行评分预测。其核心思想是将评分矩阵分解为两个低维矩阵, 即用户因子矩阵 $\mathbf{P}$ 和物品因子矩阵 $\mathbf{Q}$, 使得预测评分 $\hat{r}_{ui}$ 可通过两者点积与偏置项之和得到:

$$\hat{r}_{ui} = \mu + b_u + b_i + \mathbf{p}_u^\top \mathbf{q}_i$$

其中, μ 为全局平均评分, b_u 和 b_i 分别为用户和物品的偏置, $\mathbf{p}_u$ 和 $\mathbf{q}_i$ 分别是用户 u 和物品 i 的潜在因子向量。

模型的训练使用随机梯度下降 (SGD) 优化目标函数:

$$\min_{\mathbf{P}, \mathbf{Q}, b_u, b_i} \sum_{(u, i) \in \mathcal{K}} (r_{ui} - \hat{r}_{ui})^2 + \lambda (\|\mathbf{p}_u\|^2 + \|\mathbf{q}_i\|^2 + b_u^2 + b_i^2)$$

其中, $\mathcal{K}$ 表示训练集中的评分对, λ 为正则化系数。

实验中, 我们默认使用以下超参数: 潜在因子数量 $k = 100$, 迭代次数 $n = 20$, 学习率 $\eta = 0.005$, 因子正则化参数 $\lambda = 2.0$ 。

(五) 深度学习**1. NFC**

神经协同过滤 (Neural Collaborative Filtering, NCF) 方法, 旨在利用深度学习技术改进传统协同过滤 (Collaborative Filtering, CF) 方法, 特别是在处理隐式反馈 (如点击、购买等二元行为数据) 的推荐场景。

- 核心问题：协同过滤的非线性建模

传统协同过滤 (Collaborative Filtering, CF) 技术, 如矩阵分解 (Matrix Factorization, MF), 通过将用户和物品映射到一个共享的潜在空间, 并使用**内积 (dot product)** 作为交互函数来预测评分。然而, 作者指出:

内积模型为**线性组合**, 不能捕捉更复杂的用户-物品偏好关系;

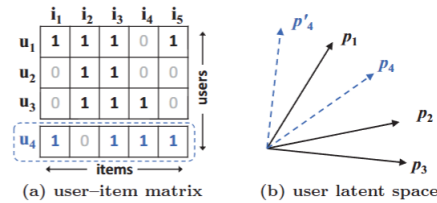


图 2: 内积模型的缺陷

图中示例说明了矩阵分解 (MF) 的局限性。从数据矩阵 (a) 中可知, 用户 u_4 与 u_1 最相似, 其次是 u_3 , 最后是 u_2 。然而在潜在空间 (b) 中, 将 P_4 (用户 u_4 的潜在向量) 放置在离 P_1 (用户 u_1 的潜在向量) 最近的位置, 会导致 P_4 离 P_2 (用户 u_2 的潜在向量) 比离 P_3 (用户 u_3 的潜在向量) 更近, 从而产生较大的排序损失。深度神经网络具备逼近任意连续函数的能力, 因此可用于学习更复杂、更强表达能力的**非线性交互函数**。

- NCF 框架设计

- (1) 整体架构

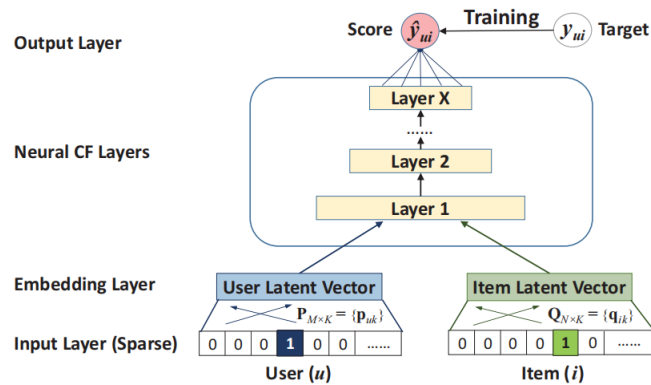


图 3: Neural matrix factorization model

1. 输入层: 用户/物品的 One-hot 编码 (可扩展至内容特征)。
2. 嵌入层 (Embedding Layer): 将稀疏向量映射为稠密潜在向量 (用户向量 $\mathbf{p}_u$, 物品向量 $\mathbf{q}_i$)。
3. 神经 CF 层: 多层级联结构, 学习用户-物品交互模式。
4. 输出层: 预测得分 $\hat{y}_{ui}$ (通过 Sigmoid 映射为概率)。

- (2) 概率化训练

- **目标函数**：二元交叉熵损失 (Log Loss)，将推荐视为二分类问题：

$$L = - \sum_{(u,i)} [y_{ui} \log \hat{y}_{ui} + (1 - y_{ui}) \log(1 - \hat{y}_{ui})]$$

- **负采样**：每个正样本采样 3-6 个负样本。

(3)NCF 的三大实例化

1. 广义矩阵分解 (GMF)：

- 替换内积为 **元素积 + 权重向量**：

$$\hat{y}_{ui} = \sigma(\mathbf{h}^T(\mathbf{p}_u \odot \mathbf{q}_i))$$

- 优势：扩展 MF 至非线性 (Sigmoid 激活)，学习隐维重要性 ($\mathbf{h}$ 可学习)。

在实现中，GMF 部分采用了用户与物品嵌入向量的逐元素乘积构成特征表示：

Listing 1: GMF 部分代码

```
1 gmf_user = self.user_emb_gmf(user)
2 gmf_item = self.item_emb_gmf(item)
3 gmf_out = gmf_user * gmf_item
```

2. 多层感知机 (MLP)：

- 拼接用户/物品向量 → MLP 学习高阶交互：

$$\phi^{MLP} = \text{MLP}([\mathbf{p}_u; \mathbf{q}_i]), \quad \hat{y}_{ui} = \sigma(\mathbf{h}^T \phi^{MLP})$$

- 关键设计：塔式结构 (高层神经元减少)、ReLU 激活 (防梯度饱和)。

MLP 分支通过拼接用户与物品向量，输入至 ReLU 激活的多层感知机中以学习高阶交互：

Listing 2: MLP 部分代码

```
1 mlp_user = self.user_emb_mlp(user)
2 mlp_item = self.item_emb_mlp(item)
3 mlp_out = self.mlp(torch.cat([mlp_user, mlp_item], dim=-1))
```

网络结构为塔式收缩，隐藏层维度设置为 [64, 32, 16]，符合论文结构设计。

3. 神经矩阵分解 (NeuMF)：

- 融合 GMF 与 MLP：

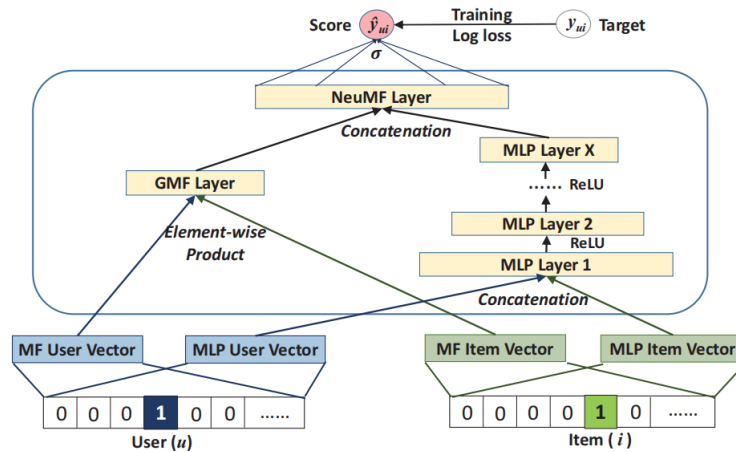


图 4: Neural matrix factorization model

- 独立学习 GMF 和 MLP 的嵌入向量 ($\mathbf{p}_u^G, \mathbf{p}_u^M$)。
- 拼接二者输出:

$$\hat{y}_{ui} = \sigma(\mathbf{h}^T [\phi^{GMF}; \phi^{MLP}])$$

Listing 3: 融合与输出

```

1 concat = torch.cat([gmf_out, mlp_out], dim=-1)
2 output = self.final(concat)

```

- 预训练策略:
 - (a) 单独训练 GMF 和 MLP 至收敛。
 - (b) 初始化 NeuMF 参数 (输出层权重按 α 加权融合)。
 - (c) 微调用 SGD (非 Adam, 避免动量干扰)。

2. DeepFM

DeepFM (Deep Factorization Machine) 是一种将因子分解机 (FM) 与深度神经网络 (DNN) 结合的推荐系统模型, 能够同时建模低阶和高阶的特征交互。其设计旨在克服传统模型 (如线性模型、FM 和 Wide&Deep) 在特征组合能力或人工特征工程上的局限性, 实现自动化的特征表达学习。

模型的整体输出由三部分组成:

$$\hat{y} = y_{\text{linear}} + y_{\text{FM}} + y_{\text{DNN}}$$

其中:

- y_{linear} 表示一阶特征贡献, 类似于线性回归;
- y_{FM} 表示二阶特征交叉建模;
- y_{DNN} 表示由深度网络学习得到的高阶非线性交互。

DeepFM (Deep Factorization Machine) 旨在同时解决以下两个问题:

- 传统线性模型（如 LR）只能建模一阶特征贡献，缺乏交叉建模能力；
- 因子分解机（FM）虽能自动建模二阶特征交互，但无法表达更复杂的非线性交互；
- 深度神经网络（DNN）具备建模高阶交互的能力，但需要人工设计特征组合，效率低下。

因此，DeepFM 设计为一个统一模型框架，融合了线性部分、FM 部分和 DNN 部分，能够端到端地建模多阶特征交互，提升推荐系统的表达能力。

图 5 展示了 DeepFM 模型的整体结构。该模型由三条路径组成，分别对应线性部分（红色权重连接）、嵌入交叉部分（黑色连接，代表 FM 与 DNN 分支共享的嵌入向量），以及深度神经网络部分（多层感知机）。底部的稀疏输入特征经过嵌入映射后，统一进入 FM 与 DNN 分支，分别建模低阶与高阶交互；同时，原始输入也进入线性通道以保留一阶特征贡献。最终三个分支的输出结果合并得到模型的评分预测结果。该结构实现了端到端的联合建模，并避免了人工特征工程，是推荐系统中常用的高效特征交叉学习框架。

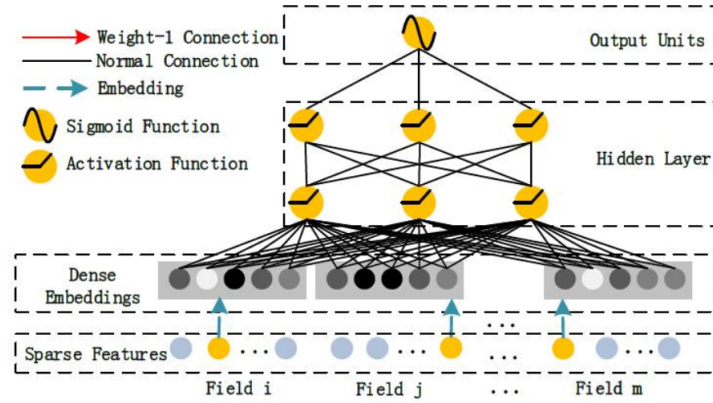


图 5: Neural matrix factorization model

1. **稀疏特征与嵌入表示**: 模型输入为多个字段的稀疏离散特征 $\{x_1, x_2, \dots, x_n\}$ ，每个字段 x_i 通过嵌入矩阵映射为低维向量 $\mathbf{v}_i \in \mathbb{R}^k$ 。所有嵌入构成矩阵：

$$X = [x_1 \mathbf{v}_1, x_2 \mathbf{v}_2, \dots, x_n \mathbf{v}_n] \in \mathbb{R}^{n \times k}$$

这些嵌入同时被送入 FM 分支和 DNN 分支。

2. **线性部分**: 原始稀疏特征输入直接用于一阶加权和建模：

$$y_{\text{linear}} = \sum_{i=1}^n w_i x_i$$

3. **FM 分支**: 利用所有嵌入之间的内积构造二阶交互项：

$$y_{\text{FM}} = \sum_{1 \leq i < j \leq n} \langle \mathbf{v}_i, \mathbf{v}_j \rangle x_i x_j$$

为提升效率可重写为：

$$y_{\text{FM}} = \frac{1}{2} \left(\left\| \sum_{i=1}^n x_i \mathbf{v}_i \right\|^2 - \sum_{i=1}^n x_i^2 \|\mathbf{v}_i\|^2 \right)$$

4. **DNN 分支（高阶交叉建模）**：将所有嵌入拼接为一个向量输入多层感知机（MLP）：

$$\mathbf{z}_0 = [x_1 \mathbf{v}_1; x_2 \mathbf{v}_2; \dots; x_n \mathbf{v}_n]$$

经激活函数 σ （如图中黄色节点所示）逐层传播：

$$\mathbf{z}_l = \sigma(W_l \mathbf{z}_{l-1} + b_l)$$

最终得到 DNN 输出：

$$y_{\text{DNN}} = W_{\text{out}} \mathbf{z}_L + b_{\text{out}}$$

5. **最终输出与训练目标**：将三部分结果相加形成最终预测评分 $\hat{y}$ ，并采用均方误差作为训练损失：

$$\hat{y} = y_{\text{linear}} + y_{\text{FM}} + y_{\text{DNN}}, \quad \mathcal{L} = \frac{1}{N} \sum_{i=1}^N (\hat{y}_i - y_i)^2$$

DeepFM 在推荐系统中的应用主要是通过将用户 ID 与物品 ID 映射为嵌入向量，结合 FM 和 DNN 模块共同建模用户与物品之间的偏好关系，从而实现评分预测。模型可直接接收原始稀疏特征，端到端输出预测评分，无需人工构造特征交叉，适合大规模推荐场景。

四、实验结果及分析

（一）模型性能对比

上面多种算法的实验结果如表1所示。可以看出，Baseline-CF 模型在所有指标上表现最好，其次是 FunkSVD 和 Baseline 模型。其他模型如 SlopeOne、DeepFM 和 NFC 的性能相对较差，而基础的协同过滤系列模型的表现明显不如前几种模型。

经过查阅文献，目前的主流 SOTA 方法都是深度学习方向的模型。但是在本次实验中，所给的数据集太少了，而且比较简单，没有过多的特征，因此深度学习模型的效果并没有明显优于传统的协同过滤方法。可以看出，数据集的规模和复杂度对模型性能有很大影响。

同样也是因为数据集的复杂程度小，并且没有过多的协变量，导致矩阵分解的方法并不如预期的好。FunkSVD 模型虽然在 MSE 和 RMSE 上表现较好，但在 MAE 上却不如 Baseline-CF 模型。这说明在处理简单数据集时，传统的协同过滤方法仍然具有一定的优势。

由于数据集的简单原因，此时的基线预测模型本身就能达到较好的效果，再加上协同过滤的特性，能够较好地捕捉用户和物品之间的关系，因此 Baseline-CF 模型在所有指标上都表现优异。

（二）在 Baseline-CF 方法中不同相似度对比

本部分，我们对比性能最优的 Baseline-CF 模型在不同相似度计算方法下的表现。结果如表2和图6所示。

表 1: 不同推荐模型性能对比

模型	MSE	RMSE	MAE
Baseline-CF	276.8952	16.6402	12.6974
SVD	299.1828	17.2969	13.2237
SVD++	302.0609	17.3799	13.4521
FunkSVD	293.1474	17.1215	13.1122
Baseline	304.5793	17.4522	13.4556
SlopeOne	308.9671	17.5775	13.4687
DeepFM	333.9574	18.2745	14.2371
NFC	346.1344	18.6047	14.3145
KNN_means_punish	346.8207	18.6231	14.2458
KNN_means	347.4640	18.6404	14.2627
KNN_item	370.7694	19.2554	14.6797
KNN_item	433.1893	20.8132	15.7766

表 2: 不同相似度方法的性能比较

相似度方法	RMSE	MAE
皮尔逊相关系数	17.4247	13.3903
调整余弦相似度	16.7993	12.7994
IUF 加权	16.7803	12.8370
余弦相似度	16.6344	12.7202
Jaccard 相似度	16.6300	12.6987

相似度方法性能比较

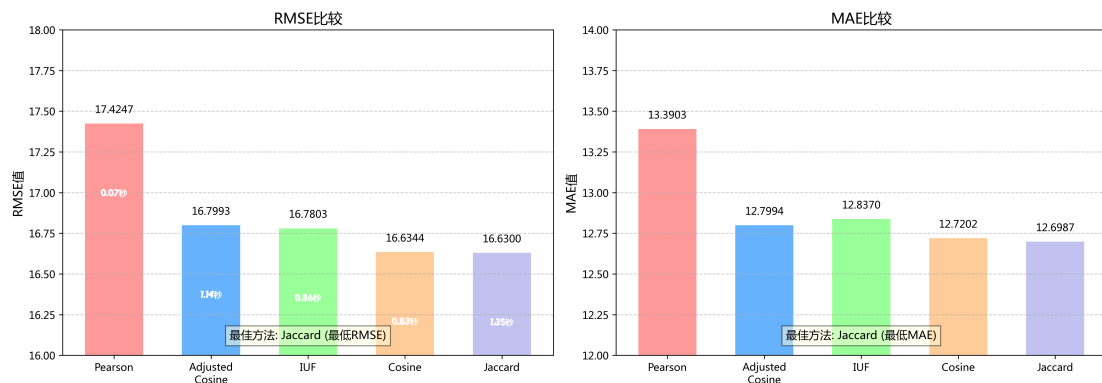


图 6: 不同相似度方法的性能比较

比较出乎意料的是，结果最好的居然是 Jaccard 相似度，而皮尔逊相关系数最差。

对于这个结果，我们猜测可能是因为训练的时候对数据进行了归一化，而没有评分的位置又为 0，导致在计算其他方式的相似度的时候，分数比较低或者交集比较小的两个物品的相似度被高估了，导致进行加权的时候选取的物品并没有那么相似。

但是，对于 Jaccard 相似度来说，就不用考虑这种情况，它只考虑是否有评分的相似度。

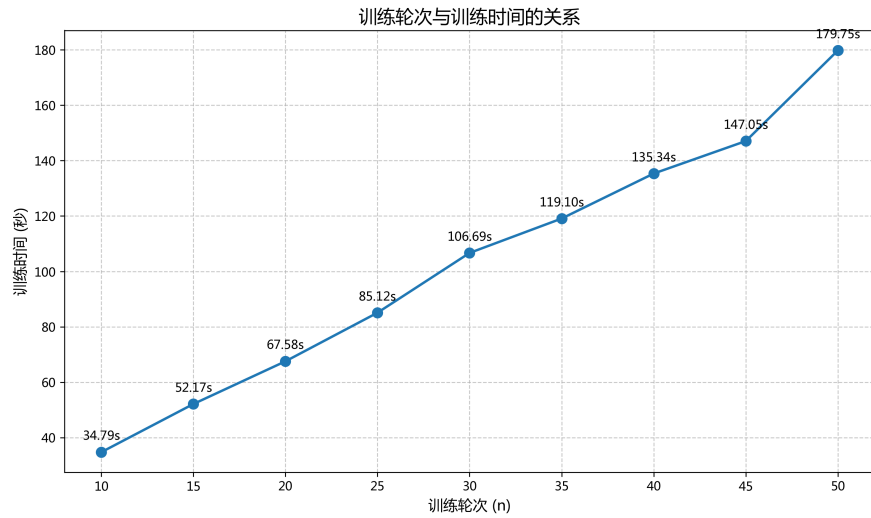


图 7: 轮次-时间

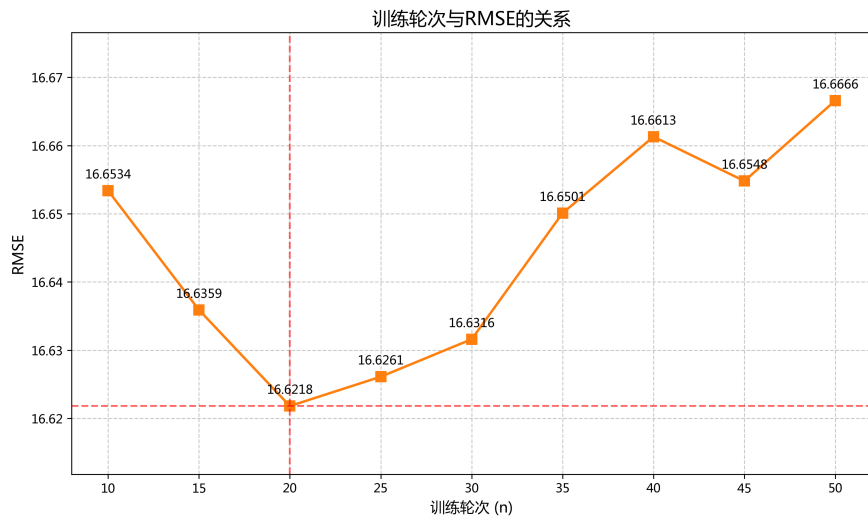


图 8: 轮次-RMSE

但这里可能会出现交集多，但是评分相反的情况，两个物品被多个人同时评分，可能两个在极端，所以在后续的算法改进中，我们可以先考虑用 Jaccard 先筛选出一批物品和某个物品比较相似，然后在用其他的相似度计算来进一步筛选，避免评分相反和相似度不准确的情况出现。

（三） Baseline-CF 方法时间分析

这部分实验我们用 Jaccard 相似度进行实验，参数详情见代码。

运行时间如图7所示，可以看到训练时间基本是线性增长（挺显而易见的，随轮次线性增长）。

而 RMES 的变化趋势随图8所示。可以发现，当训练轮次超过 20 之后，开始出现过拟合的情况，RMSE 开始上升。

(四) Baseline-CF 方法内存分析

这部分实验我们用 Jaccard 相似度进行实验，参数详情见代码。基于内存使用数据，我们可以得出以下关键发现：

- 初始内存占用：约 111.88 MB
- 数据加载增加：约 44.72 MB (训练集 + 测试集)
- 模型训练增加：约 442.4 MB
- 最终内存占用：约 593.32 MB (总增加 481.44 MB)

(五) 内存占用分布

分析显示，内存占用主要集中在以下方面：

- 相似度矩阵：约 373.49 MB (占总增加内存的 77.6%)
- 用户-物品矩阵：约 38.05 MB
- 预测结果存储：约 5.5 MB

相似度矩阵计算是内存占用的主要来源，这与算法需要存储所有物品对之间的相似度有关，空间复杂度为 $O(m^2)$ ，其中 m 为物品数量。

后续优化可以考虑一方面用稀疏矩阵存储数据和矩阵；另一方面用队列的方式存储相似度的计算结果，避免存储全部数据。或者还可以进一步用降维的思想（比如 PAC）来减少相似度矩阵大小。

五、 总结

本实验通过对推荐系统的多种算法进行实现与性能对比，探讨了不同方法在评分预测任务中的表现。实验结果表明，基线预测与协同过滤相结合的混合推荐算法（Baseline-CF）在所有指标上表现最佳，尤其是在数据稀疏性较高的情况下，能够有效提升预测精度。Baseline-CF 模型在均方误差（MSE）、均方根误差（RMSE）和平均绝对误差（MAE）等指标上均优于其他模型，其中 RMSE 为 **16.6402**，MAE 为 **12.6974**，显著优于其他传统和深度学习模型。

此外，实验发现不同相似度计算方法对模型性能有显著影响，其中 Jaccard 相似度表现最佳，进一步提升了 Baseline-CF 模型的预测精度。相比之下，皮尔逊相关系数的效果较差，可能与数据归一化处理有关。

对于深度学习模型（如 NFC 和 DeepFM），尽管其理论上具有更强的表达能力，但在本实验数据集较小且特征较少的情况下，性能未能超越传统方法。这表明数据规模和复杂度对深度学习模型的效果有重要影响。

实验还分析了模型的时间和内存开销，发现相似度矩阵的计算是内存占用的主要来源。未来优化方向包括使用稀疏矩阵存储数据、队列式存储相似度结果，以及采用降维技术减少矩阵大小。

总体而言，本实验验证了 Baseline-CF 模型在推荐系统中的有效性，并为推荐系统模型的选择和优化提供了重要参考，同时揭示了数据特性对模型性能的影响，为后续研究提供了方向。